The Frohmanian Symplectic Tether $\mathfrak{T}_F$: Written Proofs Companion to the Lean 4 Formalization

Benjamin Stanley Frohman
frohmanbenjamin@gmail.com
ORCID 0009-0006-7068-3718

October 29, 2026; revised November 1, 2026. Not peer reviewed.

Abstract

DOI: 10.5281/zenodo.23942464

This is a sibling paper to the Lean 4 repository `frohmanian-symplectic-tether`. Each result is given a written proof and cross-referenced to the Lean declaration that kernel-checks it. Result 1 is geometric: under three admissibility conditions, the symmetric correction $\mathfrak{T}_F[\kappa, w, u](F, G) = -(\kappa w)\langle \Pi_u F, \Pi_u G\rangle$ is unique up to scale, and Route A fixes the scale. Result 2 is about a *modified* system, the tethered vorticity equation $\partial_t \omega + (u \cdot \nabla)\omega = (\omega \cdot \nabla)u + \nu\Delta\omega - \kappa P(|\omega|^2\omega)$. For that system we prove the L^2 bound, the H^1 estimate, the log-loss comparison and global smoothness, *conditional on cited propositions that are listed explicitly.* **None of this resolves the Clay Millennium problem.** Plain Navier–Stokes is unchanged (Section 6). *Build status:* `lake build` of the full `TetherMayForm` library succeeds, and the axiom report (`AxiomAudit.lean`, run in CI; see the final appendix) checks 308 named theorem declarations, and the full report (`AxiomReport.lean`) covers all 382 theorems of the library; none uses `sorryAx` and none uses an axiom beyond `propext`, `Classical.choice`, `Quot.sound`. "Proved" below means kernel-checked in that sense; "conditional" means the result takes a named `Prop` hypothesis that is not proved here; "model only" means the result concerns the modified (tethered) system, not plain Navier–Stokes.

Status legend. Every result below carries one of three labels. Proved: kernel-checked in Lean with standard axioms only. Conditional: the Lean statement takes a named cited proposition (a `Prop` hypothesis) that is not proved here. Model only: the result is about the tethered system (1), not about plain Navier–Stokes. The open statement of the programme is the Frohmanian Tether Hypothesis (A)/(B); it is not proved here, and the Clay Millennium problem remains open.

1 Physical motivation

The starting picture is physical. The vortex-stretching term $(\omega \cdot \nabla)u$ behaves as if a hidden geometric constraint, an "invisible object", were shaping three-dimensional vorticity. The programme has two steps. (i) *Derive* what such an object can be from symmetry and energy considerations alone, in Hamiltonian/Lie–Poisson language. (ii) *Insert* the derived object into the dynamics, at vorticity level, through the Leray projection P.

Two pictures from the author's June 22, 2026 compilation guide the derivation. Both are formalized:

- **Chevalley–Eilenberg 2-cocycle.** Antisymmetric corrections to a Lie–Poisson bracket are CE 2-cochains. Jacobi requires them to be cocycles, and $d \circ d = 0$ makes coboundaries automatic cocycles. A *symmetric* correction is not a CE cochain at all. That is why the tether is metriplectic and not Hamiltonian (Prop. 2.7).
- $\mathbb{T}^3$ **shear.** The shear $(\sin y, 0, 0)$ is divergence free, so it is admissible. The variant $(\sin x, 0, 0)$ is not (its divergence is 1 at the origin). This is the counterexample that shows the divergence-free constraint is essential (Prop. 2.8); at field level it is essential but not sufficient for Jacobi (Theorem 2.9).

Whether real fluids carry the resulting term is *not* decided here (Section 6).

2 Result 1: geometric uniqueness of $\mathfrak{T}_F$

Throughout, H is a real inner-product space with $\dim H \geq 3$, $u \in H$, $u \neq 0$, and $\Pi_u x = x - \frac{\langle u,x\rangle}{\|u\|^2} u$.

Definition 2.1. $\mathfrak{T}_F[\kappa, w, u](F, G) := -(\kappa w)\langle \Pi_u F, \Pi_u G\rangle$.
Lean: `Frohman.Tether.frohmanianTether;frohmanianTether_apply`.

Lemma 2.2 (Basic properties)**.** *(a) $\mathfrak{T}_F$ is symmetric. (b) $\mathfrak{T}_F(F, F) \leq 0$ whenever $\kappa w \geq 0$. (c) It is degenerate with respect to the kinetic-energy gradient: $\mathfrak{T}_F(F, u) = 0$ for all F. (d) It is invariant under every isometry commuting with Π_u.*

Proof. (a) is symmetry of the inner product. (b) gives $-\kappa w\|\Pi_u F\|^2$. (c) holds because $\Pi_u u = 0$. (d) If A is an isometry with $A\Pi_u = \Pi_u A$, then $\langle \Pi_u AF, \Pi_u AG\rangle = \langle A\Pi_u F, A\Pi_u G\rangle = \langle \Pi_u F, \Pi_u G\rangle$. □

Lean: `frohmanianTether_symm`, `frohmanianTether_self_nonpos`, `degeneracy_verification`, `frohmanianTether_invariant`, `piU_comm_isometry`.

Theorem 2.3 (Uniqueness up to scale)**.** *Let B be a symmetric bilinear form that is (C1) invariant under the coadjoint isometries fixing u and (C2) degenerate with respect to u. Then $B = \mathfrak{T}_F[c, 1, u]$ for some $c \in \mathbb{R}$. If, in addition, (C3) fixes the coefficient ($C3(\mathfrak{T}_F[c, 1, u]) \Rightarrow c = \kappa w$), then $B = \mathfrak{T}_F[\kappa, w, u]$.*

Proof. (C2) and symmetry give $B(x, y) = B(\Pi_u x, \Pi_u y)$ (`degeneracy_forces_projection`). So B is determined by its restriction to $u^\perp$, which has dimension ≥ 2. Fix a unit $e_0 \perp u$ (`exists_unit_orthogonal`). For every unit $e \perp u$, (C1) supplies an isometry fixing u that sends $e_0 \mapsto e$, so $B(e, e) = B(e_0, e_0)$ (`diag_eq_of_norm_eq`). By homogeneity, $B(x, x) = B(e_0, e_0)\|\Pi_u x\|^2$. Polarization recovers $B(x, y) = B(e_0, e_0)\langle \Pi_u x, \Pi_u y\rangle$, that is, $B = \mathfrak{T}_F[c, 1, u]$ with $c = -B(e_0, e_0)$. The calibrated statement then follows from (C3). □

Lean: `admissible_eq_smul_frohmanianTether`, `uniqueness_of_minimal_metric_tether`, `uniqueness_of_minimal_metric_tether_iff`.

The calibrated statement, verbatim from `TetherMayForm/Basic.lean` (Proved):

```
theorem uniqueness_of_minimal_metric_tether [FiniteDimensional ℝ H]
    (hdim : 3 ≤ Module.finrank ℝ H) (κ w : ℝ) (u : H) (hu : u ≠ 0)
    (C3 : LinearMap.BilinForm ℝ H → Prop)
    (hC3fix : ∀ c : ℝ, C3 (𝔗_F[c, 1, u]) → c = κ * w)
```

```
(B : LinearMap.BilinForm ℝ H) (hB : IsAdmissibleCorrection u C3 B) :
B = 𝔗_F[κ, w, u]
```

Theorem 2.4 (Route A determines κ)**.** *Let $\omega \perp u$, $\|\omega\| = 1$, $w \neq 0$. Suppose $\mathfrak{T}_F[\kappa, w, u]$ matches the stretching coefficient S in the sense $\mathfrak{T}_F[\kappa, w, u](\omega, \omega) + Sw = 0$. Then $\kappa = S$. Consequently, in $\dim \geq 3$ with $S \geq 0$, $w > 0$, a correction is admissible for Route A iff $B = \mathfrak{T}_F[\kappa, w, u]$ with $\kappa = S$.*

Proof. Since $\omega \perp u$, $\Pi_u \omega = \omega$, so $\mathfrak{T}_F(\omega, \omega) = -\kappa w \|\omega\|^2 = -\kappa w$. The matching gives $(S - \kappa)w = 0$, and $w \neq 0$ yields $\kappa = S$. Here κ is a free parameter; specific values used in earlier drafts ($\kappa = 1$, $\kappa = 3/2$) are modelling/normalization choices, not derived constants, and S is supplied, not derived. The open statement of the programme is the Frohmanian Tether Hypothesis (A)/(B) (see `NAMING.md`); it is not proved here. The equivalence combines this with Theorem 2.3. □

Lean: `routeA_determines_kappa`, `frohmanian_symplectic_tether_theorem`, `uniqueness_routeA_iff`, `frohmanianTether_routeA`.

Theorem 2.5 (Nine-term Jacobi)**.** *For a classical (Jacobi) bracket P and the tethered bracket $P + \mathfrak{T}_F$, the Jacobiator expands into nine terms. The three purely classical ones cancel by Jacobi of P, and the three purely tether terms vanish. So the Jacobiator equals the cyclic sum of the mixed terms, and Jacobi holds on a set R iff that cyclic sum vanishes on R. It vanishes when u is central for P, P is homogeneous in its first slot, and $\Pi_u P(a, b) = 0$.*

Proof. Bilinear expansion gives the nine terms (`jacobiator_nine_terms`). Terms 7–9 are $\mathfrak{T}_F$-in-$\mathfrak{T}_F$ and vanish because the tether vector field is Π_u-valued and annihilated in the second application (`tetherVec_tetherVec`, `jacobi_term7`–`jacobi_term9`). The classical triple cancels (`jacobi_classical_cancel`). Under the sufficient hypotheses each mixed term contains $\Pi_u P(\cdot, \cdot) = 0$ or $P(u, \cdot) = 0$. □

Lean: `jacobiator_eq_mixed`, `jacobi_iff_mixed_cancel`, `mixedTerm_eq_zero`, `jacobiator_eq_zero_of_central`.

Theorem 2.6 (Metriplectic split)**.** *Every bilinear form splits uniquely into symmetric and antisymmetric parts. $\mathfrak{T}_F$ is purely symmetric, and the metriplectic bracket $P + \mathfrak{T}_F$ with P antisymmetric conserves kinetic energy: $\{E, E\} = 0$ and $\mathfrak{T}_F(F, \nabla E) = 0$.*

Proof. $B = \frac{1}{2}(B + B^\top) + \frac{1}{2}(B - B^\top)$. Uniqueness holds because a form that is both symmetric and antisymmetric is zero. Energy conservation follows from $P(F, F) = 0$ and degeneracy. □

Lean: `split_unique`, `symPart_frohmanianTether`, `antiPart_frohmanianTether`, `metriplecticBracket_split`, `metriplectic_energy_conserved`.

Proposition 2.7 (Chevalley–Eilenberg)**.** *(a) $d_2 \circ d_1 = 0$ on 1-cochains of any Lie algebra. (b) Coboundaries are cocycles. (c) If 2 is invertible and the symmetric part of β is alternating, it is zero. So a nonzero symmetric correction such as $\mathfrak{T}_F$ is never a CE 2-cochain.*

Proof. (a) $d_2(d_1 f)(x, y, z)$ is $-f$ applied to the Jacobi sum $[x, [y, z]] + [y, [z, x]] + [z, [x, y]] = 0$. (b) follows from (a). (c) $s(x, y) = s(y, x)$ and $s(x, y) = -s(y, x)$ give $2s = 0$. □

Lean: `d2_d1`, `cocycle_of_coboundary`, `antiCorr_cocycle_iff`, `symPart_not_cochain`.

Proposition 2.8 ($\mathbb{T}^3$ shear)**.** $\operatorname{div}(\sin y, 0, 0) \equiv 0$*, whereas* $\operatorname{div}(\sin x, 0, 0) = \cos x$*, which equals* 1 *at the origin.*

Lean: `shear_divFree`, `shearBad_div`, `shearBad_not_divFree`.

2.1 Field-level Jacobi test for the two shears

The algebra above is finite-dimensional, so divergence-freeness never enters it. To let it enter, we work on trigonometric-polynomial vector fields on $\mathbb{T}^3$, written as finite Fourier sums $a = \sum_k \hat{a}_k e^{ik\cdot x}$ (`Fld`). On these fields we use:

- the Lie bracket $[a,b] = (a\cdot\nabla)b - (b\cdot\nabla)a$, mode by mode $[ve^{ip\cdot x}, we^{iq\cdot x}] = i\big((v\cdot q)w - (w\cdot p)v\big)e^{i(p+q)\cdot x}$ (`br`);
- the L^2 pairing $\langle a,b\rangle = (2\pi)^{-3}\int a\cdot b$ (`pr`);
- the projection $\Pi_u x = x - \frac{\langle u,x\rangle}{\langle u,u\rangle}u$ (`piF`);
- the tether $T(a,b) = -(\kappa w)\langle \Pi_u a, \Pi_u b\rangle\, u$ (`tetF`);
- the tethered bracket $[a,b] + T(a,b)$ (`brT`);
- divergence-free, meaning $k\cdot\hat{a}_k = 0$ for every mode k (`DivergenceFree`).

Scalars are complex. The real fields used below have conjugate-symmetric coefficients.

Theorem 2.9 (Field-level Jacobi)**.** *(a) The field bracket satisfies Jacobi. So the Arnold (Lie–Poisson) bracket on linear functionals is classical.*

(b) If $\langle u,u\rangle \neq 0$, the Jacobiator of the tethered bracket equals the cyclic sum of the mixed terms $[T(a,b),c] + T([a,b],c)$.

(c) Divergence-free fields along e_x (parallel shears $f(y,z)e_x$) commute pairwise. Hence at $u = (\sin y, 0, 0)$ the mixed terms cancel, and Jacobi holds, for every triple in that class.

(d) At $u = (\sin x, 0, 0)$, which is not divergence-free, take the divergence-free triple $a = b = c = (1,0,0)$. The e^{ix}, x-component coefficient of the mixed sum is $\frac{3}{2}\kappa w$. So Jacobi fails whenever $\kappa w \neq 0$.

(e) At $u = (\sin y, 0, 0)$ with the divergence-free triple $a = b = c = (0,0,\sin x)$, the $e^{i(x+y)}$, z-component coefficient is $\frac{3i}{8}\kappa w \neq 0$. So Jacobi fails here too.

Lean: `jacF_eq_zero`, `jacT_eq_mixedSum`, `br_eq_zero_of_xOnly_divFree`, `shearY_mixed_cancel`, `shearY_jacobi_holds`, `not_divFree_uX`, `shearX_mixed_coeff`, `shearX_jacobi_fails`, `shearY_transverse_coeff`, `shearY_jacobi_fails_transverse`.

Scope. Divergence-freeness is what makes the two shears differ inside the e_x-directed class. It does not give cancellation on general divergence-free triples, as (e) shows, so the tethered bracket is not Poisson even at the divergence-free shear. The witnesses use $a = b = c$, where the Jacobiator is $3\{\{a,a\},a\}$. That is nonzero only because T is symmetric (metriplectic). The link from these Fourier coefficients to the `fderiv`-based fields of `Torus.lean` is not formalized for the bracket.

3 Result 2: the tethered system

3.1 Insertion through the projection

The inserted system is

$$\partial_t \omega + (u \cdot \nabla)\omega = (\omega \cdot \nabla)u + \nu \Delta \omega - \kappa P(|\omega|^2 \omega). \tag{1}$$

At velocity level it is the forced NS equation with force $F = \operatorname{curl}^{-1}(-\kappa P(|\omega|^2\omega))$. **This is a modified Navier–Stokes system, not plain Navier–Stokes.** The quartic weight is built in, not ansatzed: $\mathfrak{T}_F[\kappa, \|\omega\|^2, u](\omega, \omega) = -\kappa\|\omega\|^4$ for $\omega \perp u$ (`quarticTether_eq`).

Theorem 3.1 (Curl of forced NS)**.** *If (u, p) is a classical smooth solution of $\partial_t u + (u \cdot \nabla)u = -\nabla p + \nu\Delta u + F$ with $\operatorname{div} u = 0$, then $\omega = \operatorname{curl} u$ satisfies $\partial_t\omega + (u\cdot\nabla)\omega = (\omega\cdot\nabla)u + \nu\Delta\omega + \operatorname{curl} F$.*

Proof. Take the curl. Mixed partials commute on smooth fields, so $\operatorname{curl}\nabla p = 0$ and $\operatorname{curl}\Delta = \Delta\operatorname{curl}$, and ∂_t commutes with curl. The identity $\operatorname{curl}((u\cdot\nabla)u) = (u\cdot\nabla)\omega - (\omega\cdot\nabla)u$ for $\operatorname{div} u = 0$ is checked componentwise. □

Lean: `curlOfForcedNS`, `curl_grad`, `curl_laplacian`, `curl_advection`, `hasDerivAt_pd_time`.

Theorem 3.2 (Biot–Savart on trigonometric polynomials)**.** *For each nonzero wavevector k and amplitude $a \perp k$, the mode $b = \frac{ik\times a}{|k|^2} e^{ik\cdot x}$ satisfies $\operatorname{curl} b = a\, e^{ik\cdot x}$ and $\operatorname{div} b = 0$, and it is the unique such divergence-free mode. Summing gives a right inverse of curl on mean-zero divergence-free trigonometric polynomials.*

Proof. $\operatorname{curl} b = \frac{ik\times(ik\times a)}{|k|^2} e^{ik\cdot x} = \frac{-(k\cdot a)k + |k|^2 a}{|k|^2} e^{ik\cdot x} = a\, e^{ik\cdot x}$, and $\operatorname{div} b \propto k\cdot(k\times a) = 0$. Curl is linear over finite sums. □

Lean: `curlC_bsC`, `divC_bsC`, `bsC_unique`, `curl_bsMode`, `div_bsMode`, `curl_bsSum`, `curlRightInverse_trig`.

Corollary 3.3 (Velocity form ⇒ vorticity form)**.** *If curl^{-1} is a right inverse of curl on the admissible class containing $-\kappa P(|\omega|^2\omega)$, then every velocity solution of the tethered NS solves (1).*

Lean: `tetheredVorticity_of_velocity`, `curl_lerayTetherForce`.

3.2 L^2 (enstrophy) bound

Lemma 3.4. *If P is a symmetric idempotent and $P\omega = \omega$, then $\langle \omega, -\kappa P g\rangle = -\kappa\langle\omega, g\rangle$. With $g = |\omega|^2\omega$ this equals $-\kappa\|\omega\|_4^4$.*

Lean: `inner_leray_of_divFree`, `tether_L2_term`, `inner_cubic`.

Theorem 3.5. *Let $E = \frac{1}{2}\|\omega\|_2^2$ and $Q = \|\omega\|_4^4$. Assume $E' = S - \nu D - \kappa Q$ (Cited Prop. 1), $S \le C\sqrt{2E}\sqrt{Q}$ (Cited Prop. 2), and $(2E)^2 \le Q$ (Cited Prop. 3). Then for $\kappa > 0$, $E(t) \le \max(E_0,\, C^2/(2\kappa^2))$ on $[0, T]$.*

Proof. Young with $\lambda = \kappa/2$: $S \le \frac{\kappa}{2}Q + \frac{C^2\cdot 2E}{2\kappa}$. Dropping $-\nu D \le 0$ and using $Q \ge (2E)^2$ gives the logistic inequality $E' \le \frac{C^2}{\kappa}E - 2\kappa E^2$ (`enstrophy_deriv_logistic`), so $E' < 0$ whenever $E > C^2/(2\kappa^2)$. A barrier argument (`image_le_of_deriv_right_lt_deriv_boundary'`) with a level M' slightly above the maximum, followed by letting $M' \downarrow \max$, gives the bound. □

Lean: `young_lam`, `stretch_absorb`, `enstrophy_deriv_le`, `enstrophy_deriv_logistic`, `tethered_enstrophy_uniform`, `tethered_L2_vorticity_bounded`.

3.3 H^1 (palinstrophy) estimate

Lemma 3.6 (Tether is sign-definite at H^1)**.** *For $\omega \in \mathbb{R}^3$ and $D = \nabla\omega$, $\sum_{ij} D_{ij}\,\partial_j(|\omega|^2\omega_i) = |\omega|^2|D|^2 + \frac{1}{2}\sum_j(2\sum_k \omega_k D_{kj})^2 \geq 0$, with no alignment assumption.*

Lean: `tether_H1_density_eq`, `tether_H1_density_nonneg`, `tether_H1_term`.

Theorem 3.7. *Let $G = \|\nabla\omega\|_2^2$, $L = \|\Delta\omega\|_2$. Assume $G' = 2(-\nu L^2 - \kappa J + N)$ with $J \geq 0$ (Cited Prop. 4) and $N \leq A\,G^{1/4}L^{3/2}$ (Cited Prop. 5). Then $G(t) \leq G(0)\exp(\frac{27A^4}{16\nu^3}t)$.*

Proof. Drop $-\kappa J \leq 0$. Young with exponents $(4, 4/3)$: $AG^{1/4}L^{3/2} \leq \nu L^2 + \frac{27A^4}{32\nu^3}G$ (`young_43`, `amgm_four`). So $G' \leq \frac{27A^4}{16\nu^3}G$, and Grönwall (`le_gronwallBound_of_liminf_deriv_right_le`) finishes. □

Lean: `palin_deriv_le`, `tethered_palinstrophy_gronwall`, `higherNormControl_tethered`.

3.4 Log-loss comparison and global smoothness

At a spatial maximum of $|\omega|$, with $y = \|\omega\|_\infty$, the maximum principle gives a nonpositive viscous term (proved: `viscousAtMax_of_max`, `periodic_exists_max`). The tether gives $-\kappa y^3$ (proved from $\mathfrak{T}_F$: `tetherFeedback_of_quartic`). With Cited Props. 7, 8, 9 this gives $y' \leq C(1 + Z_0)\,y + C\log(e + Z)\,y^2 - (\kappa/2 - C_{abs})y^3$ (`tether_pointwise_logloss`). The cubic dominates, so y stays below the largest root of the comparison field (`comparison_bound_logloss`).

Theorem 3.8 (Global smoothness of the tethered system, conditional)**.** *Assume the tethered log inequality (Cited Prop. 10), higher-norm control, BKM continuation (Cited Prop. 11) and local continuation (Cited Prop. 12). Then the solution is smooth on $[0, T]$ for every T. Higher-norm control is itself discharged by Theorems 3.5 and 3.7 together with Cited Prop. 6.*

Proof. Let $\sigma = \sup\{T : \text{smooth on } [0, T]\}$, assuming this set is bounded. On $[0, \sigma)$ the comparison bound gives a uniform $\sup y$. BKM continuation then gives smoothness on $[0, \sigma]$, and local continuation extends it past σ, a contradiction. □

Lean: `global_regularity_logloss`, `global_regularity`, `lemma_3_1_uniform_bound_and_continuation`, `frohmanian_symplectic_tether_theorem_global`.

3.5 Remaining cited propositions (explicit hypotheses, never axioms)

Each item below is a `Prop` hypothesis in Lean. No `axiom` or `sorry` is used.

Cited Proposition 1 (`TetheredEnstrophyIdentity`)**.** Integrated enstrophy identity for (1). It follows from multiplying by ω, integrating by parts on $\mathbb{T}^3$, and the lemma above.

Cited Proposition 2 (`StrainL2Bound`)**.** $\int \omega \cdot S\omega \leq C\|\omega\|_2\|\omega\|_4^2$: Calderón–Zygmund / Biot–Savart, classical.

Cited Proposition 3 (`L4L2Holder`)**.** $\|\omega\|_2^4 \leq \|\omega\|_4^4$ on the unit-volume torus (Hölder). **Now proved** for every probability space with $E = \frac{1}{2}\int g$, $Q = \int g^2$, $g = |\omega|^2 \in L^2$ (`l4l2Holder_of_prob`, via nonnegativity of variance).

Cited Proposition 4 (`TetheredH1Identity`)**.** The H^1 energy identity (Majda–Bertozzi [4] §3.2 for the NS part).

Cited Proposition 5 (`NonlinH1Bound`)**.** Sobolev/Gagliardo–Nirenberg bound of the nonlinear term [2, 3].

Cited Proposition 6 (`H1ToHsBootstrap`)**.** H^1 bounds on $[0, T)$ imply H^s bounds for all s. For plain NS this is classical parabolic regularity [2, 3, 4]. **The tether-specific step (controlling $\kappa P(|\omega|^2\omega)$ in H^s, which is plausible since $H^2 \subset L^\infty$ in 3D) is not in the literature and remains to be proved.**

Cited Proposition 7 (`EnergyIdentity`, `StretchingByStrain`)**.** Pointwise evolution at the maximum, and stretching $\le \|S\|_\infty y^2$.

Cited Proposition 8 (`BKMLogInequality`)**.** $\|S\|_\infty \le C(1 + \|\omega\|_2 + \|\omega\|_\infty \log(e + \|\omega\|_{H^s}))$ [1]; see also Brezis–Gallouët and Kozono–Taniuchi [5].

Cited Proposition 9 (`CrossControlled`)**.** The cross term at the maximum is absorbed with constant C_{abs}.

Cited Proposition 10 (`NSTetherLogInequality`, `NSTetherEnstrophyInequality`)**.** The integrated (time-continuous) form of the pointwise inequality along the solution.

Cited Proposition 11 (`BKMContinuation`)**.** Beale–Kato–Majda [1]: bounded $\|\omega\|_\infty$ on $[0, T)$ implies continuation. Stated for plain NS; its extension to (1) is assumed.

Cited Proposition 12 (`LocalContinuation`)**.** Local existence for smooth periodic data (Fujita–Kato [6], Kato [7]).

Cited Proposition 13 (`CurlRightInverse` beyond trigonometric polynomials)**.** Proved on trigonometric polynomials. The extension to the smooth admissible class requires a density/-continuity argument not yet formalized.

Why the rest remain hypotheses (what Mathlib lacks, v4.30.0-rc1). `TetheredEnstrophyIdentity` and `TetheredH1Identity` need a solution theory: a time-dependent smooth periodic solution of (1), differentiation under the integral sign over $\mathbb{T}^3$, and integration by parts on the torus for vector fields. Mathlib has `AddCircle` Haar measure and Fourier series, but no PDE solution concept or torus Green's identities. `StrainL2Bound` needs the L^2 isometry $\|\nabla u\|_2 = \|\omega\|_2$ for Biot–Savart (Parseval on $\mathbb{T}^3$) and Hölder in L^4. Mathlib has Parseval for `AddCircle` in one variable, but no multivariable torus Sobolev/Biot–Savart theory. `NonlinH1Bound` needs Sobolev embedding $H^1 \subset L^6$ and Gagliardo–Nirenberg on $\mathbb{T}^3$. Neither is in Mathlib. `BKMContinuation`, `BKMLogInequality`, `LocalContinuation` and `H1ToHsBootstrap` need Calderón–Zygmund theory, Brezis–Gallouët, and Fujita–Kato local well-posedness. None is in Mathlib, and the tether-specific H^s step is open mathematics. The `CurlRightInverse` extension needs density of trigonometric polynomials in the smooth admissible class with C^∞ convergence of Biot–Savart. Mathlib has the L^2 density of Fourier modes, not the smooth-topology statement on $\mathbb{T}^3$.

Search of the author's own prior work for $H^1 \to H^2$. The author's earlier work was searched: unpublished drafts and code from June–October 2026 and the `Clay-` repository (`latex/section10_gaps.tex`, Oct. 7, 2026). All invoke "BKM + parabolic regularity bootstrap" as a *cited classical black box* (in one case a `sorry`). None contains an independent proof of the $H^1 \to H^2$ step, let alone the tether-specific one. These are cited as the provenance of the plan, not as a proof.

4 Riccati comparison: plain versus tethered, and its source

Provenance. The comparison ODE $y' = Cy^2 - \kappa'' y^3$ was introduced by the author, B. S. Frohman. It appears in his unpublished draft dated 2026-05-21; its earliest git timestamp is 2026-06-19 in an unpublished repository of the author. Details are in `paper/publication_note_riccati_provenance.md`. The bound it gives applies only to the tethered comparison ODE below, not to plain Navier–Stokes.

Plain Riccati. For $C, y_0 > 0$, $y(t) = y_0/(1 - Cy_0 t)$ solves $y' = Cy^2$, $y(0) = y_0$. It exceeds every bound before $T^* = 1/(Cy_0)$.
Lean: `ricSol_hasDerivAt`, `riccati_blowup`.

Tethered. If $y' \le Cy^2 - \kappa'' y^3$ with $\kappa'' > 0$ and $y(0) \ge 0$, then $y(t) \le \max(y(0), C/\kappa'')$ for all t.
Lean: `tethered_ceiling`, `comparison_bound`.

Section 3 uses the same mechanism at the enstrophy level, $E \le \max(E_0, C^2/(2\kappa^2))$ (`tethered_enstrophy_uniform`). Both statements are scalar ODE facts about the tethered model. They say nothing about plain Navier–Stokes.

5 Metriplectic (Morrison) reading of the field-level test

Split the bracket as $\{\cdot,\cdot\} = [\cdot,\cdot] + T$. In a metriplectic system, Jacobi is required only of the antisymmetric part. The symmetric part must instead be symmetric, semidefinite and degenerate.

1. *Antisymmetric part.* The Lie bracket satisfies Jacobi on all trigonometric-polynomial fields (`jacF_eq_zero`).

2. *Symmetric part.* $T(a,b) = T(b,a)$ (`pr_comm,tetF_symm`). Degeneracy: for $H = \frac{1}{2}\langle u, u\rangle$ we have $dH = u$, and $\Pi_u u = 0$, so $T(u,\cdot) = T(\cdot,u) = 0$ (`tetF_energy_left,tetF_energy_right`). The quadratic form is $d(a) = -\kappa w \langle \Pi_u a, \Pi_u a\rangle$.

3. *Shears.* At $u = (\sin y, 0, 0)$ with $a = (0, 0, \sin x)$, $d = -\kappa w/2$. At $u = (\sin x, 0, 0)$ with $a = (1, 0, 0)$, $d = -\kappa w$. Both are real and ≤ 0 when $\kappa w \ge 0$, and both vanish in the energy direction $a = u$ (`dissF_shearY,dissF_shearX,dissF_shear_nonpos,dissF_energy_uY,dissF_energy_uX`).

The checks come out differently for the two shears. $(\sin y, 0, 0)$ is divergence free and passes all of them that were tested: symmetry, energy degeneracy, and the sign at the tested direction. $(\sin x, 0, 0)$ passes the same algebraic checks, but it is not divergence free (`not_divFree_uX`), so it is not an admissible state. The nonzero mixed sums of Section 3 ($\frac{3}{2}\kappa w$ and $\frac{3i}{8}\kappa w$) mean the bracket is not Lie–Poisson. They do not contradict the metriplectic structure, because the metriplectic axioms do not ask the full bracket to satisfy Jacobi. *Sign condition for all real fields (proved).* Call a field real when $\hat{a}_{-k} = \overline{\hat{a}_k}$. For real fields, $\langle a, b\rangle$ is real (`conj_pr`). $\langle a, a\rangle = \sum_k |\hat{a}_k|^2 \ge 0$ (`pr_self_eq,pr_self_re_nonneg`). Π_u maps real fields to real fields (`isRealF_piF`). Hence $d(a) = -\kappa w \langle \Pi_u a, \Pi_u a\rangle$ is real and ≤ 0 for all real u, a when $\kappa w \ge 0$ (`dissF_nonpos`).

Entropy/Casimir degeneracy (not proved; reason). The metriplectic axioms also need a Casimir S of the classical bracket with $\{S,\cdot\} = 0$. In this model, `br` is the Lie bracket of vector fields and $\langle\cdot,\cdot\rangle$ is the plain L^2 pairing. The Lie–Poisson Casimir of ideal fluids is helicity, and

its Casimir property uses the coadjoint (Leray-projected) pairing and Biot–Savart *inside* the bracket. Neither is formalized here, so we make no claim. The strongest true statement we have is local: divergence-free parallel shears commute, so $[u, \cdot]$ vanishes on the shear class at $u = (\sin y, 0, 0)$ (`br_eq_zero_of_xOnly_divFree`).

Where 4/3 *appears.* The coefficients in the bracket test are $\frac{3}{2}$ and $\frac{3i}{8}$. The exponent 4/3 does not appear there. It appears in the estimate chain, at the H^1 rung: the Gagliardo–Nirenberg/Sobolev bound $N \le A\, G^{1/4} L^{3/2}$ is followed by Young's inequality with exponents $(4, 4/3)$ (Section 3; `PalinstrophyH1`). Gagliardo–Nirenberg interpolation also appears in the author's May 2026 chain, as $M_\varepsilon \le C_{\mathrm{GN}} \|\omega_\varepsilon\|_{L^6}^{3/2} \|\omega_\varepsilon\|_{L^2}^{1/2} + \dots$ (Full Living Document).

6 Open question, stated plainly

1. **Plain Navier–Stokes is unchanged.** For a velocity field that solves plain NS, nothing above adds a damping term. The author's October notes say so in these words: "Because $B(F, H) = 0$ by Proposition [prop:degen], the tether does not add a damping term to the PDE (eq:vort) itself" (author's unpublished notes, October 2026); "A degeneracy $B(F, H) \equiv 0$ on a kinetic-energy Hamiltonian leaves the energy field classical and does not insert $-\kappa M^2$ into the strain" and "Equation (eq:missing) is not an identity of the unmodified vorticity equation" (author's unpublished note, 7 Oct. 2026); "Derivation of the rate from the unmodified vorticity equation remains open" (author's unpublished status note, 10 Oct. 2026). (Bracketed labels inside quotations are the cited files' own equation references.) The mechanism is therefore the *degeneracy* $\Pi_u u = 0$, not a statement that the Leray projection annihilates a velocity-level cubic source; those notes make no such claim. Here the cubic term is inserted by hand at vorticity level through P, which is why the results concern (1) only. The Lean predicates `IsTrueNS` and `IsTetheredNS` coincide only at $\kappa = 0$ (`tetheredNS_zero_iff`), and at $\kappa = 0$ no feedback is realized (`not_tetherRealizesFeedback_zero`).

2. **The Clay Millennium problem remains open.** Nothing in this paper or repository claims otherwise.

3. **Do real fluids carry this term?** Answering this would need either a derivation of $-\kappa P(|\omega|^2 \omega)$ from plain NS (none is known) or experimental/numerical evidence. Result 1 says that *if* such a symmetric, invariant, energy-degenerate correction exists, it must be $\mathfrak{T}_F$.

Claim-to-Lean table

Labels as in the status legend: Conditional marks results whose Lean statement takes a cited proposition as an explicit hypothesis; Model only marks results about the tethered system (1).

Claim	Lean declarations	Status
Basic properties	`frohmanianTether_symm`, `frohmanianTether_self_nonpos`, `degeneracy_verification`, `frohmanianTether_invariant`, `piU_comm_isometry`	Proved
Uniqueness up to scale	`admissible_eq_smul_frohmanianTether`, `uniqueness_of_minimal_metric_tether`, `uniqueness_of_minimal_metric_tether_iff`	Proved
Route A determines κ	`routeA_determines_kappa`, `frohmanian_symplectic_tether_theorem`, `uniqueness_routeA_iff`, `frohmanianTether_routeA`	Proved
Nine-term Jacobi	`jacobiator_eq_mixed`, `jacobi_iff_mixed_cancel`, `mixedTerm_eq_zero`, `jacobiator_eq_zero_of_central`	Proved
Metriplectic split	`split_unique`, `symPart_frohmanianTether`, `antiPart_frohmanianTether`, `metriplecticBracket_split`, `metriplectic_energy_conserved`	Proved
Chevalley–Eilenberg	`d2_d1`, `cocycle_of_coboundary`, `antiCorr_cocycle_iff`, `symPart_not_cochain`	Proved
$\mathbb{T}^3$ shear	`shear_divFree`, `shearBad_div`, `shearBad_not_divFree`	Proved
Field-level Jacobi	`jacF_eq_zero`, `jacT_eq_mixedSum`, `br_eq_zero_of_xOnly_divFree`, `shearY_mixed_cancel`, `shearY_jacobi_holds`, `not_divFree_uX`, `shearX_mixed_coeff`, `shearX_jacobi_fails`, `shearY_transverse_coeff`, `shearY_jacobi_fails_transverse`	Proved
Curl of forced NS	`curlOfForcedNS`, `curl_grad`, `curl_laplacian`, `curl_advection`, `hasDerivAt_pd_time`	Proved
Biot–Savart on trigonometric polynomials	`curlC_bsC`, `divC_bsC`, `bsC_unique`, `curl_bsMode`, `div_bsMode`, `curl_bsSum`, `curlRightInverse_trig`	Proved
Velocity form $\Rightarrow$ vorticity form	`tetheredVorticity_of_velocity`, `curl_lerayTetherForce`	Proved; Model only
L^2 tether term (lemma)	`inner_leray_of_divFree`, `tether_L2_term`, `inner_cubic`	Proved; Model only
L^2 enstrophy bound	`young_lam`, `stretch_absorb`, `enstrophy_deriv_le`, `enstrophy_deriv_logistic`, `tethered_enstrophy_uniform`, `tethered_L2_vorticity_bounded`	Conditional; Model only
Tether is sign-definite at H^1	`tether_H1_density_eq`, `tether_H1_density_nonneg`, `tether_H1_term`	Proved; Model only
H^1 palinstrophy estimate	`palin_deriv_le`, `tethered_palinstrophy_gronwall`, `higherNormControl_tethered`	Conditional; Model only

Claim	Lean declarations	Status
Global smoothness of the tethered system, conditional	`global_regularity_logloss`, `global_regularity`, `lemma_3_1_uniform_bound_and_continuation`, `frohmanian_symplectic_tether_theorem_global`	Conditional; Model only

A Build and axiom audit

The formal development was built and audited as follows. This is a build and axiom audit, not a certificate.

- Repository:
  https://github.com/BenFrohman/frohmanian-symplectic-tether
- Verified commit:
  `77f02ae94c433c1893be1c34fa7fabe6419c17f2`
- Lean toolchain: `leanprover/lean4:v4.30.0-rc1`
- Mathlib: tag `v4.30.0-rc1`, commit (from `lake-manifest.json`)
  `0c154d67103f74be3a0f2c509f72ccbf5be9f2a7`
- Continuous integration run (build, axiom audit, `sorry`/`axiom` scan; passed):
  https://github.com/BenFrohman/frohmanian-symplectic-tether/actions/runs/41795567086
- SHA-256 of the GitHub source tarball of the verified commit, downloaded twice with identical result:
  https://github.com/BenFrohman/frohmanian-symplectic-tether/archive/77f02ae94c433c1893be1c34fa7fabe6419c17f2.tar.gz
  `e643778a642701419b26967169cf4acd615df18dfe9bf42fcf28cc8823cebba7`

The tarball hash refers to the verified commit rather than to a release tag, since a document cannot contain the hash of an archive that includes itself. The Lean sources are unchanged between that commit and the release tag.

The development contains no `sorry` and no user-declared axioms. Every one of the 308 theorems and lemmas listed below depends only on Lean's standard axioms `propext`, `Classical.choice` and `Quot.sound` (or on none). The table is the verbatim output of `lake env lean AxiomAudit.lean` (`#print axioms`) on the verified commit.

Lean declaration	`#print axioms`
`Frohman.Tether.IsAdmissibleCorrection.c1`	`[propext, Classical.choice, Quot.sound]`
`Frohman.Tether.IsAdmissibleCorrection.c2`	`[propext, Classical.choice, Quot.sound]`
`Frohman.Tether.IsAdmissibleCorrection.c3`	`[propext, Classical.choice, Quot.sound]`
`Frohman.Tether.IsAdmissibleCorrection.symm`	`[propext, Classical.choice, Quot.sound]`

Lean declaration	#print axioms
Frohman.Tether.ProducesControllableNegativeFeedback.calibrated	[propext, Classical.choice, Quot.sound]
Frohman.Tether.ProducesControllableNegativeFeedback.nonpos	[propext, Classical.choice, Quot.sound]
Frohman.Tether.admissible_eq_smul_frohmanianTether	[propext, Classical.choice, Quot.sound]
Frohman.Tether.degeneracy_forces_projection	[propext, Classical.choice, Quot.sound]
Frohman.Tether.degeneracy_verification	[propext, Classical.choice, Quot.sound]
Frohman.Tether.degeneracy_verification_smul	[propext, Classical.choice, Quot.sound]
Frohman.Tether.diag_eq_of_norm_eq	[propext, Classical.choice, Quot.sound]
Frohman.Tether.exists_unit_orthogonal	[propext, Classical.choice, Quot.sound]
Frohman.Tether.feedback_fixes_coefficient	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanianTether_admissible	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanianTether_apply	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanianTether_degenerate	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanianTether_energy_neutral	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanianTether_feedback	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanianTether_invariant	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanianTether_self_nonpos	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanianTether_symm	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanian_symplectic_tether_theorem_calibrated	[propext, Classical.choice, Quot.sound]
Frohman.Tether.gauge_uniqueness_weak	[propext, Classical.choice, Quot.sound]
Frohman.Tether.inner_piU_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.inner_u_piU	[propext, Classical.choice, Quot.sound]
Frohman.Tether.piU_apply	[propext, Classical.choice, Quot.sound]
Frohman.Tether.piU_apply_eq_zero_iff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.piU_comm_isometry	[propext, Classical.choice, Quot.sound]
Frohman.Tether.piU_eq_self_of_inner_eq_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.piU_smul_self	[propext, Classical.choice, Quot.sound]
Frohman.Tether.projectionOrthogonalToU	[propext, Classical.choice, Quot.sound]
Frohman.Tether.tetheredBracket_kineticEnergy	[propext, Classical.choice, Quot.sound]

Lean declaration	#print axioms
Frohman.Tether.uniqueness_of_minimal_metric_tether	[propext, Classical.choice, Quot.sound]
Frohman.Tether.uniqueness_of_minimal_metric_tether_iff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.DFMode.hK	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.DFMode.ha	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.DFMode.hb	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.bsC_unique	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.bsSum_diff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.comp_mode	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.curlC_bsC	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.curlRightInverse_trig	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.curl_bsMode	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.curl_bsSum	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.divC_bsC	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.div_bsMode	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.gradL2_mode	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.kSq_ne_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.mode_contDiff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.mode_diff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.pd_comp_mode	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.pd_trig	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.th_e	[propext, Classical.choice, Quot.sound]
Frohman.Tether.BiotSavart.toE_apply	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CE.antiCorr_alt	[propext, Quot.sound]
Frohman.Tether.CE.antiCorr_cocycle_iff	[propext, Quot.sound]
Frohman.Tether.CE.cocycle_of_coboundary	[propext, Quot.sound]
Frohman.Tether.CE.d2_d1	[propext, Quot.sound]
Frohman.Tether.CE.symPart_not_cochain	[propext, Quot.sound]
Frohman.Tether.Shear.div_mode	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Shear.shearBad_div	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Shear.shearBad_not_divFree	[propext, Classical.choice, Quot.sound]

Lean declaration	#print axioms
Frohman.Tether.Shear.shear_divFree	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CitedProps.instIsProbabilityMeasureForallFinOfNatNatRealTorusMeasure	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CitedProps.l4l2Holder_of_prob	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CitedProps.l4l2Holder_torus	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CitedProps.sq_integral_le	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.comp_add_diff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.comp_adv	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.comp_curl	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.comp_grad	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.comp_laplacian	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.curlOfForcedNS	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.curl_add	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.curl_advection	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.curl_apply	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.curl_grad	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.curl_laplacian	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.curl_lin	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.dif	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.fderiv_slice	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.hasDerivAt_pd_time	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.hasDerivAt_slice	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.hasDerivAt_vec3	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.lap_scalar_sub	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.pd3	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.pd_add	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.pd_comm	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.pd_comp	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.pd_comp'	[propext, Classical.choice, Quot.sound]

Lean declaration	#print axioms
Frohman.Tether.CurlNS.pd_comp_adv	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.pd_const_smul	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.pd_lap	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.pd_mul	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.pd_sub	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.pd_sum	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.sm_comp	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.sm_curl	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.sm_pd	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.vec3_apply	[propext, Classical.choice, Quot.sound]
Frohman.Tether.CurlNS.vec3_eq	[propext, Classical.choice, Quot.sound]
Frohman.Tether.EnstrophyL2.enstrophy_deriv_le	[propext, Classical.choice, Quot.sound]
Frohman.Tether.EnstrophyL2.enstrophy_deriv_logistic	[propext, Classical.choice, Quot.sound]
Frohman.Tether.EnstrophyL2.stretch_absorb	[propext, Classical.choice, Quot.sound]
Frohman.Tether.EnstrophyL2.tethered_L2_vorticity_bounded	[propext, Classical.choice, Quot.sound]
Frohman.Tether.EnstrophyL2.tethered_enstrophy_gronwall	[propext, Classical.choice, Quot.sound]
Frohman.Tether.EnstrophyL2.tethered_enstrophy_uniform	[propext, Classical.choice, Quot.sound]
Frohman.Tether.EnstrophyL2.young_lam	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanianTether_orth_self	[propext, Classical.choice, Quot.sound]
Frohman.Tether.nsOnly_not_sufficient	[propext, Classical.choice, Quot.sound]
Frohman.Tether.tetherFeedback_of_frohmanianTether	[propext, Classical.choice, Quot.sound]
Frohman.Tether.tetherTerm_of_frohmanianTether	[propext, Classical.choice, Quot.sound]
Frohman.Tether.young_kappa_quarter	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.br_add_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.br_add_right	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.br_antisymm	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.br_eq_zero_of_xOnly_divFree	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.br_self	[propext, Classical.choice, Quot.sound]

Lean declaration	#print axioms
Frohman.Tether.FieldJacobi.br_single	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.br_smul_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.br_uX_eX_coeff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.br_uY_aZ_coeff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.br_zero_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.br_zero_right	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.bvec_zero_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.bvec_zero_right	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.divFree_aZ	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.divFree_eX	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.divFree_smul	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.divFree_uY	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.jacF_add_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.jacF_cycle	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.jacF_eq_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.jacF_single	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.jacF_zero_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.jacT_eq_mixedSum	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.kx_ne	[propext]
Frohman.Tether.FieldJacobi.mixedF_eq_zero_of_shear	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.mixedSumF_diag	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.mixedSumF_eq_zero_of_shear	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.ne_kx	[propext]
Frohman.Tether.FieldJacobi.ne_ky	[propext]
Frohman.Tether.FieldJacobi.not_divFree_uX	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.piF_add	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.piF_of_pr_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.piF_smul_self	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.piF_zero	[propext, Classical.choice, Quot.sound]

Lean declaration	#print axioms
Frohman.Tether.FieldJacobi.pr_aZ_aZ	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_add_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_add_right	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_eX_eX	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_single	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_smul_right	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_uX_eX	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_uX_uX	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_uY_aZ	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_uY_uY	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_zero_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.pr_zero_right	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.shearX_jacobi_fails	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.shearX_mixed_coeff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.shearX_mixed_ne_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.shearY_jacobi_fails_transverse	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.shearY_jacobi_holds	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.shearY_mixed_cancel	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.shearY_transverse_coeff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.shearY_transverse_ne_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.tetF_add_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.tetF_tetF	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.tetF_zero_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.xOnly_eX	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.xOnly_smul	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.xOnly_uX	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldJacobi.xOnly_uY	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.IsRealF.neg'	[propext, Classical.choice, Quot.sound]

Lean declaration	#print axioms
Frohman.Tether.FieldMetriplectic.conj_pr	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.dissF_energy_uX	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.dissF_energy_uY	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.dissF_nonpos	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.dissF_shearX	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.dissF_shearY	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.dissF_shear_nonpos	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.isRealF_piF	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.isRealF_sub_smul	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.mem_support_neg	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.piF_self	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.pr_comm	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.pr_eq_sum	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.pr_self_eq	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.pr_self_re_nonneg	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.pr_uX_uX	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.pr_uY_uY	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.tetF_energy_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.tetF_energy_right	[propext, Classical.choice, Quot.sound]
Frohman.Tether.FieldMetriplectic.tetF_symm	[propext, Classical.choice, Quot.sound]
Frohman.Tether.inner_secondDeriv_le_of_isMaxOn	[propext, Classical.choice, Quot.sound]
Frohman.Tether.inner_secondDeriv_nonpos_of_isMaxOn	[propext, Classical.choice, Quot.sound]
Frohman.Tether.periodic_exists_max	[propext, Classical.choice, Quot.sound]
Frohman.Tether.periodic_int_shift	[propext, Classical.choice, Quot.sound]
Frohman.Tether.quarticTether_eq	[propext, Classical.choice, Quot.sound]
Frohman.Tether.quarticWeight_of_def	[propext, Classical.choice, Quot.sound]
Frohman.Tether.secondDeriv_nonpos_of_isLocalMax	[propext, Classical.choice, Quot.sound]
Frohman.Tether.tetherFeedback_of_quartic	[propext, Classical.choice, Quot.sound]

Lean declaration	#print axioms
Frohman.Tether.viscousAtMax_of_max	[propext, Classical.choice, Quot.sound]
Frohman.Tether.viscous_nonpos_at_max	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanian_symplectic_tether_theorem_global	[propext, Classical.choice, Quot.sound]
Frohman.Tether.JacobiWitness.crossBr_isClassical	[propext, Classical.choice, Quot.sound]
Frohman.Tether.JacobiWitness.mixed_sum_eq	[propext, Classical.choice, Quot.sound]
Frohman.Tether.JacobiWitness.mixed_sum_ne_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.JacobiWitness.tethered_jacobiator_ne_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.IsClassicalBracket.add_left	[propext, Classical.choice, Quot.sound]
Frohman.Tether.IsClassicalBracket.jacobi	[propext, Classical.choice, Quot.sound]
Frohman.Tether.jacobi_classical_cancel	[propext, Classical.choice, Quot.sound]
Frohman.Tether.jacobi_iff_mixed_cancel	[propext, Classical.choice, Quot.sound]
Frohman.Tether.jacobi_term7	[propext, Classical.choice, Quot.sound]
Frohman.Tether.jacobi_term8	[propext, Classical.choice, Quot.sound]
Frohman.Tether.jacobi_term9	[propext, Classical.choice, Quot.sound]
Frohman.Tether.jacobiator_eq_mixed	[propext, Classical.choice, Quot.sound]
Frohman.Tether.jacobiator_eq_zero_of_central	[propext, Classical.choice, Quot.sound]
Frohman.Tether.jacobiator_nine_terms	[propext, Classical.choice, Quot.sound]
Frohman.Tether.mixedTerm_eq_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.tetherVec_tetherVec	[propext, Classical.choice, Quot.sound]
Frohman.Tether.tetheredBracket_nested	[propext, Classical.choice, Quot.sound]
Frohman.Tether.comparisonField_equilibrium	[propext, Classical.choice, Quot.sound]
Frohman.Tether.comparisonField_neg_above	[propext, Classical.choice, Quot.sound]
Frohman.Tether.comparisonField_pos_between	[propext, Classical.choice, Quot.sound]
Frohman.Tether.comparisonField_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.comparison_bound	[propext, Classical.choice, Quot.sound]
Frohman.Tether.comparison_bound_of_gt	[propext, Classical.choice, Quot.sound]
Frohman.Tether.layer2_bound	[propext, Classical.choice, Quot.sound]
Frohman.Tether.young_absorption	[propext, Classical.choice, Quot.sound]

Lean declaration	#print axioms
Frohman.Tether.comparison_bound_logloss	[propext, Classical.choice, Quot.sound]
Frohman.Tether.global_regularity_logloss	[propext, Classical.choice, Quot.sound]
Frohman.Tether.logField_neg_above	[propext, Classical.choice, Quot.sound]
Frohman.Tether.logThreshold_nonneg	[propext, Classical.choice, Quot.sound]
Frohman.Tether.tether_pointwise_logloss	[propext, Classical.choice, Quot.sound]
Frohman.Tether.RouteAFeedback.leadingCoefficient_eq_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanian_symplectic_tether_theorem	[propext, Classical.choice, Quot.sound]
Frohman.Tether.kappa_eq_stretching_of_admissible	[propext, Classical.choice, Quot.sound]
Frohman.Tether.routeA_determines_kappa	[propext, Classical.choice, Quot.sound]
Frohman.Tether.routeA_layer2_bound	[propext, Classical.choice, Quot.sound]
Frohman.Tether.antiPart_antisymm	[propext, Classical.choice, Quot.sound]
Frohman.Tether.antiPart_apply	[propext, Classical.choice, Quot.sound]
Frohman.Tether.antiPart_frohmanianTether	[propext, Classical.choice, Quot.sound]
Frohman.Tether.antiPart_self	[propext, Classical.choice, Quot.sound]
Frohman.Tether.degenerate_of_parts	[propext, Classical.choice, Quot.sound]
Frohman.Tether.eq_zero_of_symm_of_antisymm	[propext, Classical.choice, Quot.sound]
Frohman.Tether.metriplecticBracket_split	[propext, Classical.choice, Quot.sound]
Frohman.Tether.metriplectic_energy_conserved	[propext, Classical.choice, Quot.sound]
Frohman.Tether.parts_degenerate	[propext, Classical.choice, Quot.sound]
Frohman.Tether.split_unique	[propext, Classical.choice, Quot.sound]
Frohman.Tether.symPart_add_antiPart	[propext, Classical.choice, Quot.sound]
Frohman.Tether.symPart_apply	[propext, Classical.choice, Quot.sound]
Frohman.Tether.symPart_frohmanianTether	[propext, Classical.choice, Quot.sound]
Frohman.Tether.symPart_self	[propext, Classical.choice, Quot.sound]
Frohman.Tether.symPart_symm	[propext, Classical.choice, Quot.sound]
Frohman.Tether.sym_anti_pairing_eq_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.symm_antisymm_pairing_eq_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.PalinstrophyH1.amgm_four	[propext, Classical.choice, Quot.sound]

Lean declaration	#print axioms
Frohman.Tether.PalinstrophyH1.higherNormControl_tethered	[propext, Classical.choice, Quot.sound]
Frohman.Tether.PalinstrophyH1.lp_absorb	[propext, Classical.choice, Quot.sound]
Frohman.Tether.PalinstrophyH1.palin_deriv_le	[propext, Classical.choice, Quot.sound]
Frohman.Tether.PalinstrophyH1.tether_H1_density_eq	[propext, Classical.choice, Quot.sound]
Frohman.Tether.PalinstrophyH1.tether_H1_density_nonneg	[propext, Classical.choice, Quot.sound]
Frohman.Tether.PalinstrophyH1.tethered_palinstrophy_gronwall	[propext, Classical.choice, Quot.sound]
Frohman.Tether.PalinstrophyH1.young_43	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Riccati.ricSol_hasDerivAt	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Riccati.ricSol_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Riccati.riccati_blowup	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Riccati.tethered_ceiling	[propext, Classical.choice, Quot.sound]
Frohman.Tether.RouteAFeedback.jacobi_reduced	[propext, Classical.choice, Quot.sound]
Frohman.Tether.RouteAFeedback.matches_stretching	[propext, Classical.choice, Quot.sound]
Frohman.Tether.RouteAFeedback.maximizer_reduced	[propext, Classical.choice, Quot.sound]
Frohman.Tether.RouteAFeedback.maximizer_unit	[propext, Classical.choice, Quot.sound]
Frohman.Tether.RouteAFeedback.nonpos	[propext, Classical.choice, Quot.sound]
Frohman.Tether.RouteAFeedback.poisson_unchanged	[propext, Classical.choice, Quot.sound]
Frohman.Tether.RouteAFeedback.toControllable	[propext, Classical.choice, Quot.sound]
Frohman.Tether.frohmanianTether_routeA	[propext, Classical.choice, Quot.sound]
Frohman.Tether.routeA_fixes_coefficient	[propext, Classical.choice, Quot.sound]
Frohman.Tether.routeA_iff_controllable_on_tether	[propext, Classical.choice, Quot.sound]
Frohman.Tether.uniqueness_routeA_iff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.NSTetherSolution.smoothOn_mono	[propext, Classical.choice, Quot.sound]
Frohman.Tether.NSTetherSolution.smoothOn_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.NSTetherSolution.y_zero_nonneg	[propext, Classical.choice, Quot.sound]
Frohman.Tether.comparison_majorant_global_bound	[propext, Classical.choice, Quot.sound]
Frohman.Tether.global_regularity	[propext, Classical.choice, Quot.sound]
Frohman.Tether.lemma_3_1_uniform_bound_and_continuation	[propext, Classical.choice, Quot.sound]

Lean declaration	#print axioms
Frohman.Tether.Torus.classicalSolutionOn_mono	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Torus.classicalSolutionOn_of_nonpos	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Torus.classicalSolutionOn_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Torus.not_tetherRealizesFeedback_zero	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Torus.tetheredNS_zero_iff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Torus.toNSTetherSolution_smoothOn_iff	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Torus.toNSTetherSolution_y	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Torus.toNSTetherSolution_y_nonneg	[propext, Classical.choice, Quot.sound]
Frohman.Tether.Torus.vortSup_nonneg	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.LerayProj.idem	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.LerayProj.symm	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.curl_lerayTetherForce	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.enstrophyIdentity_of_vorticityLevel	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.inner_cubic	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.inner_leray_of_divFree	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.quarticTether_density	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.tether_H1_density_nonneg'	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.tether_H1_term	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.tether_L2_term	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.tetheredVorticity_of_velocity	[propext, Classical.choice, Quot.sound]
Frohman.Tether.VorticityTether.tetheredVorticity_zero_iff	[propext, Classical.choice, Quot.sound]

References

[1] J. T. Beale, T. Kato, A. Majda, Remarks on the breakdown of smooth solutions for the 3-D Euler equations, *Comm. Math. Phys.* 94 (1984) 61–66.

[2] R. Temam, *Navier–Stokes Equations and Nonlinear Functional Analysis*, 2nd ed., SIAM, 1995.

[3] P. Constantin, C. Foias, *Navier–Stokes Equations*, Univ. Chicago Press, 1988.

[4] A. J. Majda, A. L. Bertozzi, *Vorticity and Incompressible Flow*, Cambridge, 2002.

[5] H. Kozono, Y. Taniuchi, Limiting case of the Sobolev inequality in BMO, *Comm. Math. Phys.* 214 (2000) 191–200.

[6] H. Fujita, T. Kato, On the Navier–Stokes initial value problem I, *Arch. Rational Mech. Anal.* 16 (1964) 269–315.

[7] T. Kato, Strong L^p-solutions of the Navier–Stokes equation in $\mathbb{R}^m$, *Math. Z.* 187 (1984) 471–480.